

tf.transpose Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/transpose

Transposes a , where a is a Tensor.

Function Signatures

```
tf.transpose( a, perm=None, conjugate=False, name='transpose'
)
```

Code Examples

```
tf.transpose(
a, perm=None, conjugate=False, name='transpose'
)
```

```
tf.transpose(
a, perm=None, conjugate=False, name='transpose'
)
```

```
x = tf.constant([[1, 2, 3], [4, 5, 6]])
tf.transpose(x)
```

```
x = tf.constant([[1, 2, 3], [4, 5, 6]])
```

```
tf.transpose(x)
```

```
array([[1, 4],
```

Documentation

Transposes a, where a is a Tensor.

Used in the notebooks

- Matrix approximation with Core APIs
 - Effective Tensorflow 2
 - Better performance with tf.function
 - Advanced automatic differentiation
 - TensorFlow basics
 - Scalable model compression
 - Time series forecasting
 - Client-efficient large-model federated learning via `federated_select`` and sparse aggregation
 - Quantum data

- Neural machine translation with a Transformer and Keras

Permutes the dimensions according to the value of perm.

The returned tensor's dimension i will correspond to the input dimension $\text{perm}[i]$. If perm is not given, it is set to $(n-1 \dots 0)$, where n is the rank of the input tensor. Hence, by default, this operation performs a regular matrix transpose on 2-D input Tensors.

If conjugate is True and `a.dtype` is either `complex64` or `complex128` then the values of `a` are conjugated and transposed.

For example:

Equivalently, you could call `tf.transpose(x, perm=[1, 0])`.

If `x` is complex, setting `conjugate=True` gives the conjugate transpose:

'perm' is more useful for n -dimensional tensors where $n > 2$:

As above, simply calling `tf.transpose` will default to `perm=[2,1,0]`.

To take the transpose of the matrices in dimension-0 (such as when you are transposing matrices where 0 is the batch dimension), you would set `perm=[0,2,1]`.

Returns

numpy compatibility

In numpy transposes are memory-efficient constant time operations as they

simply return a new view of the same data with adjusted strides.

TensorFlow does not support strides, so transpose returns a new tensor with the items permuted.